Documentation du code

inspection-acier-vision — inspection de surfaces d'acier, de la caméra au modèle en production

Ali Yaghi — octobre 2026
Dépôt : github.com/yagii99/inspection-acier-vision

Ce document décrit chaque fichier du dépôt et, pour chacun, chaque fonction, méthode ou bloc de code : ce qu'il fait, ce qu'il reçoit et ce qu'il produit. Les scripts Python sans fonctions sont décrits bloc par bloc.

Sommaire

1. Vue d'ensemble
2. 01 — Traitement classique (Python)
   - generer_tole.py · etape1_pretraitement.py · etape2_seuillage.py · etape3_morphologie.py · etape4_caracterisation.py · etape5_fft.py · etape6_limites.py
3. 01 — Traitement classique (C#) : csharp_InspectionClassique/Program.cs
4. 02 — Deep learning
   - etape3_preparer_neu.py · etape3_entrainer.py · etape4_anomalies.py · etape5_analyse_erreurs.py · etape6_bilan.py
5. 03 — Intégration C# : YoloOnnx
   - Program.cs · comparer_python.py
6. 03 — Intégration C# : InspectionWpf
   - CameraService.cs · Detecteur.cs · JournalDetections.cs · MainWindow.xaml(.cs) · App.xaml(.cs)
7. 04 — Caméra motorisée : Program.cs

fonction fonction ou méthode **bloc** bloc de code sans fonction **type** classe, record ou enum **événement** événement

1. Vue d'ensemble

```
inspection-acier-vision/
├── 01_traitement_classique/
│   ├── python/                     générer une tôle test, puis prétraiter, seuiller, nettoyer, mesurer
│   └── csharp_InspectionClassique/ la même chaîne en C# (OpenCvSharp)
├── 02_deep_learning/               NEU-DET → YOLO : préparer, entraîner, analyser, anomalies, bilan
├── 03_integration_csharp/
│   ├── YoloOnnx/                   inférence ONNX en console + comparaison avec Python
│   └── InspectionWpf/              application opérateur : caméra + modèle + journal
├── 04_camera_motorisee/            séquence d'acquisition avec une caméra motorisée
└── docs/                           images et résultats (metriques du modèle retenu)
```

Chaîne de traitement

```
Image ─► prétraitement ─► seuillage ─► morphologie ─► mesure/classement      (01, méthode classique)
NEU-DET ─► VOC→YOLO ─► entraînement ─► test + export ONNX ─► seuils par classe (02, deep learning)
Caméra ─► file ─► inférence ONNX (C#) ─► journal CSV + affichage               (03, production)
Plan de positions ─► déplacement ─► stabilisation ─► capture ─► contrôle ─► sauvegarde   (04)
```

Technologies

Outil	Utilisation
Python, NumPy, OpenCV, Matplotlib	Traitement classique, figures
Ultralytics (YOLO26 / YOLO11), PyTorch	Entraînement, évaluation, export ONNX, détection d'anomalies
C# .NET 8, WPF	Application opérateur
Basler pylon (émulateur)	Acquisition caméra, déclenchement
ONNX Runtime, OpenCvSharp	Inférence et traitement d'image en C#

2. 01 — Traitement classique (Python)

Dossier `01_traitement_classique/python`. Les scripts se lancent dans l'ordre ; chacun lit ce que produit le précédent. Résolution utilisée partout : 0,25 mm par pixel.

generer_tole.py

Rôle : fabriquer une image de tôle de test dont on connaît exactement les défauts. **Produit :** `tole.png`, `verite_terrain.json`.

bloc `Paramètres et graine`

`MM_PAR_PIXEL = 0.25`, image de 1200 × 600 px (300 × 150 mm). Graine aléatoire fixe (42) : la même image à chaque lancement.

bloc `Métal et texture de laminage`

Image grise à 140, puis ajout de stries horizontales sinusoïdales de période 6 px (broadcasting d'une colonne `[:, None]` sur toutes les colonnes).

bloc `Défauts`

Dessine trois défauts avant l'éclairage : une rayure claire inclinée (`cv2.line`), une tache sombre ronde et floue, une petite inclusion de 6 px (1,5 mm). Chaque défaut est ajouté à la liste de vérité terrain avec sa position et sa taille en mm.

bloc `Éclairage inégal et bruit`

Multiplie l'image par une pente de lumière (55 % à gauche, 100 % à droite, un peu plus sombre en bas), puis ajoute un bruit gaussien d'écart-type 7. Bornage 0–255 et conversion en `uint8`.

bloc `Enregistrement`

Écrit l'image et la vérité terrain (JSON), puis affiche la liste des défauts.

etape1_pretraitement.py

Rôle : débruiter et corriger l'éclairage. **Lit :** `tole.png`. **Produit :** `etape1_resultat.png`.

bloc `A. Débruitage`

Calcule un flou gaussien 5×5 et un flou médian 5×5 pour les comparer. Le médian retire les pixels isolés en gardant les bords nets ; c'est lui qui est utilisé ensuite.

bloc `B. Correction d'éclairage`

Estime le fond (éclairage) par un flou gaussien de sigma 40 px, puis calcule `médian / fond × 128` en `float32`. La division annule l'effet multiplicatif de la lumière. Bornage et retour en `uint8`.

fonction `niveaux(im) → (gauche, droite)`

Renvoie la moyenne des 200 premières et des 200 dernières colonnes de l'image. Sert à mesurer l'écart d'éclairage entre les deux côtés, avant et après correction.

bloc `Mesures et figure`

Affiche l'écart gauche/droite et le bruit (écart-type) dans une zone sans défaut pour l'image brute, le gaussien et le médian. Figure 3×2 : image brute, médian, fond estimé, image corrigée, profil d'une ligne, histogrammes.

etape2_seuillage.py

Rôle : séparer les défauts du métal sain en comparant quatre méthodes. **Lit :** `tole.png`. **Produit :** `etape2_resultat.png`, `masque_defauts.png`.

bloc `Prétraitement`

Reprend l'étape 1 (médian puis division par le fond).

fonction `pourcentage(masque) → float`

Part de l'image marquée comme défaut, en %. Contrôle de bon sens : les vrais défauts couvrent environ 0,3 % de l'image.

bloc `A. Seuil fixe sur l'image brute`

`cv2.threshold(img, 70, 255, THRESH_BINARY_INV)` : tout pixel sous 70 devient défaut. Échoue : le côté sous-éclairé passe sous le seuil.

bloc `B. Otsu`

Seuil automatique qui suppose un histogramme à deux bosses. Échoue : avec si peu de pixels de défaut, il n'y a qu'une bosse, qu'Otsu coupe en deux.

bloc `C. Seuil adaptatif`

Chaque pixel est comparé à la moyenne de sa fenêtre 51×51 moins 15. Ne détecte que les défauts plus sombres que leur voisinage : la rayure claire est ratée.

bloc `D. Seuil statistique à deux côtés (retenu)`

Niveau du métal = médiane de l'image ; bruit = 1,4826 × MAD (déviation absolue médiane). Un pixel est défaut si `|pixel − niveau| > 5 × bruit` (`np.where`). Détecte les défauts clairs et sombres ; médiane et MAD ne sont pas influencées par les pixels de défaut.

bloc `Résultats et figure`

Affiche le niveau, le bruit, les seuils, le seuil d'Otsu et le pourcentage marqué par chaque méthode. Enregistre le masque D et une figure avec l'histogramme (échelle log) et les quatre masques.

etape3_morphologie.py

Rôle : nettoyer le masque. **Lit :** `masque_defauts.png`. **Produit :** `etape3_operations.png`, `etape3_resultat.png`, `masque_propre.png`.

bloc `Partie 1 : image jouet`

Dessine un rectangle avec deux trous, une ligne fine et 40 points de bruit, puis applique érosion, dilatation, ouverture, fermeture et ouverture 3×3 + fermeture 15×15 avec un noyau 3×3. Figure de comparaison.

fonction `morceaux(m) → int`

Nombre de morceaux blancs séparés (composantes connexes), fond exclu (`n − 1`).

fonction `surface_rayure(m) → int`

Nombre de pixels blancs dans la zone de la rayure. Sert à vérifier qu'un traitement ne l'efface pas.

fonction `filtre_surface(m, surface_min) → masque`

Numérote les morceaux avec `connectedComponentsWithStats` et ne recopie dans une image noire que ceux dont la surface est ≥ `surface_min` . Supprime le bruit sans déformer les défauts gardés. La boucle commence à 1 pour sauter le fond (étiquette 0).

fonction `trous(m) → int`

Compte les trous noirs entièrement entourés de blanc : inverse le masque, numérote les morceaux et ne garde que ceux qui ne touchent aucun bord de l'image.

bloc `Partie 2 : comparaison et méthode retenue`

Compare ouverture 3×3 + fermeture 3×3, ouverture 7×7 (efface la rayure de 2 px), filtre par surface, et la méthode retenue : **fermeture 3×3 puis filtre par surface (12 px = 0,75 mm²)**. Tableau morceaux / trous / pixels de rayure, puis enregistrement du masque propre.

etape4_caracterisation.py

Rôle : mesurer et classer chaque défaut, comparer à la vérité terrain. **Lit :** `tole.png` , `masque_propre.png` , `verite_terrain.json` . **Produit :** `etape4_resultat.png` , `rapport_defauts.csv` .

bloc `Chargement`

Lit les images et la vérité terrain (`MM` = mm par pixel). `MARGE_BORD = 8` px : les défauts touchant le bord sont ignorés. Recalcule l'image corrigée et le niveau du métal sain (pour la polarité).

bloc `Boucle sur les défauts`

Pour chaque composante connexe :

- contour extérieur (`findContours`) et plus petit rectangle tourné (`minAreaRect`) : centre, longueur, largeur, angle ramené entre −90° et 90° ;
- surface en mm² (× MM²), longueur et largeur en mm, diamètre équivalent (disque de même surface), périmètre ;
- allongement = longueur / largeur ; circularité = 4π·surface / périmètre² (1 = disque) ;
- polarité : moyenne des pixels du défaut sur l'image corrigée, comparée au métal sain ;
- classement par règles : rayure (allongement > 5), tache (sombre, diamètre ≥ 5 mm), inclusion (sombre), sinon inconnu.

Chaque défaut est stocké dans un dictionnaire ; les clés commençant par `_` sont internes (contour, rectangle).

bloc `Comparaison à la vérité terrain`

Pour chaque vrai défaut, cherche le défaut détecté le plus proche (au-delà de 30 px : non détecté), compare la taille (écart en %) et la classe.

bloc `Rapport CSV et image annotée`

Écrit une ligne par défaut (`csv.DictWriter` , séparateur `;`). Dessine le rectangle tourné (`boxPoints` + `drawContours`) et l'étiquette de chaque défaut, et un graphique allongement / circularité qui montre la séparation des classes.

etape5_fft.py

Rôle : supprimer la texture de laminage par filtrage fréquentiel. **Lit :** `tole.png` . **Produit :** `etape5_resultat.png` , `etape5_retire.png` .

bloc `1. FFT 1D d'une colonne`

Centre une colonne sans défaut (moyenne retirée), calcule `np.fft.rfft` et repère le pic : 1/6 cycle par pixel, soit la période des stries.

bloc `2. Spectre 2D`

`fft2` de l'image centrée, `fftshift` (fréquence nulle au centre), `log1p` pour l'affichage. Les stries donnent deux points brillants à ±100 lignes du centre.

bloc `3. Filtre coupe-bande`

Masque à 1 partout, deux disques à 0 sur les pics, bords adoucis par un flou. Multiplication du spectre, transformée inverse (`ifftshift` , `ifft2` , partie réelle) et remise de la moyenne.

fonction `bruit_robuste(im) → float`

Bruit mesuré par 1,4826 × MAD, robuste aux pixels de défaut.

bloc `4. Vérifications et figures`

Compare avant/après : amplitude des stries, bruit, contraste de la tache et de la rayure (qui doivent rester inchangés). Figures du signal, des spectres, du filtre, du zoom avant/après et de ce qui a été retiré.

etape6_limites.py

Rôle : appliquer la même chaîne à une tôle plus réaliste pour montrer les limites du classique. **Produit :** `etape6_resultat.png` .

bloc `La tôle difficile`

Vrais défauts : rayure, pli léger (−9 niveaux, flou), nid de 25 piqûres groupées. Pièges : marquage « B4127 » (écrit sur une image à part puis recopié avec `img[marq > 0] = 95`) et ombre nette d'un support (bande de 40 colonnes à 72 %). Liste de vérité : nom, défaut ou non, zone.

bloc `Chaîne des étapes 1 à 4`

Médian, correction d'éclairage, seuil à 5 × bruit, fermeture 3×3, composantes connexes, filtre de surface et de bord, mesures et classement par règles, sans aucun réglage nouveau.

fonction `dans(c, z) → bool`

Vrai si le point `c` est dans la zone `z = (x0, y0, x1, y1)`.

bloc `Bilan et figure`

Pour chaque élément de la vérité : défaut détecté, raté (faux négatif), vu en plusieurs morceaux, fausse alarme ou ignoré à raison. Figure : détections en cyan, vérité en vert (bon) ou rouge (erreur), et le masque.

3. 01 — Traitement classique (C#)

csharp_InspectionClassique/Program.cs

Rôle : la chaîne des étapes 1 à 4 en C# avec OpenCvSharp. **Lancer :** `dotnet run -- tole.png`. **Produit :** `resultat_csharp.png`, `rapport_csharp.csv`.

bloc `Constantes et lecture`

`MmParPixel = 0.25`, `K = 5`, `SurfaceMin = 12`, `MargeBord = 8`. Lecture de l'image en gris, arrêt propre si elle est introuvable. Chronomètre lancé.

bloc `1. Prétraitement`

Flou médian, conversion en float, fond par flou gaussien (sigma 40), `Cv2.Divide(médian, fond, scale: 128)`.

bloc `2. Seuillage statistique`

Niveau = médiane des pixels, bruit = 1,4826 × MAD, `Absdiff` puis `Threshold` à K × bruit, conversion en masque 8 bits.

bloc `3. Nettoyage`

Fermeture avec une ellipse 3×3, puis étiquetage par `ConnectedComponentsWithStats` ; le filtre de surface est appliqué dans la boucle suivante.

bloc `4. Caractérisation`

Pour chaque composante (sauf le fond) : rejet si trop petite ou au bord ; masque du défaut (`Compare`), plus grand contour, `MinAreaRect`, mesures en mm, allongement, circularité, polarité (`Cv2.Mean` sur le masque), classement par règles. Ajout d'un `Defaut` à la liste.

bloc `Résultats`

Affiche niveau, bruit, seuils, nombre de défauts et temps de traitement ; écrit le CSV (format invariant) ; dessine les rectangles tournés (`Polylines`) et les étiquettes.

fonction `static double Mediane(float[] tableau)`

Médiane d'un tableau : copie, tri, valeur du milieu. Le tableau d'origine n'est pas modifié.

type `record Defaut(Id, Classe, Rect, SurfaceMm2, LongueurMm, LargeurMm, DiametreEqMm, Allongement, Circularite, Polarite)`

Un défaut mesuré et classé.

4. 02 — Deep learning

Dossier `02_deep_learning`. Dataset NEU-DET : 1 800 images de 200 × 200 px, 6 classes (crazing, inclusion, patches, pitted_surface, rolled-in_scale, scratches). Les scripts se lancent depuis un dossier de travail contenant `neu_yolo/`, `neu.yaml` et `runs/`.

etape3_preparer_neu.py

Rôle : convertir NEU-DET du format Pascal VOC au format YOLO et découper train/val/test. **Lancer :** `python etape3_preparer_neu.py CHEMIN_NEU`. **Produit :** `neu_yolo/`, `neu.yaml`.

bloc `Paramètres`

`CLASSES` : la liste ordonnée des 6 classes (l'ordre définit les numéros). Dossier source lu en argument, graine fixe pour un découpage reproductible.

bloc `1. Paires image + annotation`

Parcourt `ANNOTATIONS/` et `Validation_Annotations/` ; pour chaque XML, retient l'image JPG de même nom si elle existe.

fonction `vers_yolo(xml_path) → list[str]`

Lit un XML VOC (taille de l'image, puis chaque `object` avec `name` et `bndbox`). Ramène les boîtes dans l'image, ignore les boîtes vides, puis convertit les coins en pixels en `classe cx cy largeur hauteur` normalisés entre 0 et 1. Renvoie une ligne par boîte.

bloc `3. Découpage stratifié`

Range les paires par classe (classe lue dans le nom du fichier), mélange chaque classe puis la coupe en 70 / 15 / 15 %. Copie les images et écrit les TXT dans `images/` et `labels/` de chaque partie, en comptant images et boîtes.

bloc `4. neu.yaml et contrôle`

Écrit le fichier de configuration YOLO (chemin, dossiers train/val/test, noms des classes), puis affiche le nombre d'images par classe et par partie et le nombre de boîtes.

etape3_entrainer.py

Rôle : entraîner YOLO, évaluer sur le test, enregistrer les métriques, exporter en ONNX. **Lancer :** `python etape3_entrainer.py [--modele yolo26n.pt] [--imgsz 224] [--epoques 100] [--batch 32] [--nom ...]`.

fonction `arguments() → Namespace`

Définit et lit les options de la ligne de commande (`argparse`) : modèle de départ, fichier de données, taille d'image, nombre d'époques, batch, nom de l'essai.

`fonction` `main()`

- Nom de l'essai par défaut : `<modèle>_<taille>` ; un essai n'écrase jamais le précédent.
- Entraînement depuis des poids pré-entraînés : arrêt anticipé (patience 20), graine fixe, mode déterministe, flips horizontaux et verticaux, aucune rotation, aucune variation de teinte ni de saturation, luminosité ±30 %, mosaïque coupée sur les 10 dernières époques.
- Évaluation de `best.pt` (meilleure époque sur la validation) sur le jeu de **test**.
- Écriture de `metriques.json` : modèle, taille, date, mAP@0.5, mAP@0.5:0.95, précision, rappel, AP@0.5 par classe.
- Export du meilleur modèle en ONNX à la même taille d'image.

Le point d'entrée est protégé par `if __name__ == "__main__"` (nécessaire sous Windows pour les workers du DataLoader).

etape4_anomalies.py

Rôle : détection d'anomalies sans images de défauts (version simplifiée de PatchCore). **Produit :** `etape4_resultat.png`. Calcul sur GPU si disponible.

`fonction` `tole_saine(rng, marquage=False, ombre=False) → image`

Génère une tôle sans défaut avec des variations normales tirées au hasard (gris, intensité et phase des stries, éclairage, bruit). Options : un numéro de bobine imprimé à une position aléatoire, une ombre de support.

`fonction` `tole_difficile() → image`

Recrée la tôle de `etape6_limites.py` (3 défauts, 2 pièges) et l'agrandit à 608 × 1216 en recopiant les bords.

`bloc` `Réseau descripteur`

Charge `yolo11n-cls.pt` (pré-entraîné sur ImageNet), récupère ses couches PyTorch, en mode évaluation, sur le GPU. Aucun réentraînement.

`fonction` `caracteristiques(img) → tenseur (192, H/4, W/4)`

Convertit l'image en tenseur NCHW normalisé, la fait passer couche par couche jusqu'à la couche 4, garde les sorties des couches 2 (détails, 1/4) et 4 (contexte, 1/8), agrandit la seconde à la taille de la première, les concatène et fait une moyenne 3×3. Renvoie une description de 192 valeurs par patch de 4 × 4 px.

`fonction` `banque(images, par_image=3000, taille=10000) → tenseur`

Construit la banque de mémoire du « normal » : tire 3 000 patchs par image saine, puis sélectionne 10 000 patchs par coreset glouton (à chaque tour, le patch le plus éloigné de ceux déjà choisis, distances calculées sur une projection aléatoire en 64 dimensions). Couvre la variété du normal, cas rares compris.

`fonction` `carte_anomalie(img, memoire) → carte (H, W)`

Pour chaque patch, distance à son plus proche voisin dans la banque (`torch.cdist` puis minimum, par blocs de 2 048 patchs pour ne pas saturer la mémoire). Remise en grille, agrandissement à la taille de l'image, lissage.

`bloc` Les deux expériences

A : banque de 40 tôles saines simples. B : banque de tôles avec marquages (une sur deux) et ombres (une sur trois). Pour chacune, seuil = 99,9e percentile des scores de 6 tôles saines de validation, carte de la tôle difficile, et verdict par zone (score maximal au-dessus du seuil = anomalie). Figure avec les contours au-dessus du seuil.

etape5_analyse_erreurs.py

Rôle : comprendre les erreurs d'un modèle et choisir un seuil par classe. **Lancer :** `python etape5_analyse_erreurs.py [chemin/best.pt]`. **Produit :** deux galeries JPG, `seuils_par_classe.json`.

`fonction` `iou(a, b) → float`

Intersection sur union de deux boîtes `(x0, y0, x1, y1)` : surface commune / (surface A + surface B − surface commune). 0 si elles ne se touchent pas.

`fonction` `lire_annotations(chemin_label, w, h) → list[(classe, boîte)]`

Lit un fichier d'annotation YOLO et reconvertit chaque ligne (centre et taille normalisés) en coins en pixels.

`fonction` `predire(modele, dossier_images, conf) → dict`

Fait prédire le modèle sur toutes les images du dossier (une à la fois, `stream=True`), retrouve le fichier d'annotation de chaque image, et renvoie `{nom: (image, annotations, prédictions)}` avec des prédictions `(classe, score, boîte)`.

`fonction` `apparier(annotations, predictions, seuils=None) → list[événements]`

Filtre éventuellement les prédictions par seuil de leur classe, les trie par score décroissant, puis associe chacune à l'annotation libre qui la recouvre le mieux. IoU ≥ 0,5 : bonne détection ou mauvaise classe ; sinon fausse alarme. Chaque annotation n'est prise qu'une fois ; celles restées libres sont des défauts ratés.

`fonction` `galerie(cas, noms, titre_fichier, n=12)`

Fabrique une mosaïque de 12 vignettes 300 × 300 (annotations en vert, prédictions en rouge, bandeau de texte), 4 par ligne, et l'enregistre.

`fonction` `dernier_modele() → str`

Chemin du `best.pt` le plus récent dans `runs/detect` ; arrêt avec un message clair s'il n'y en a aucun.

`fonction` `main()`

- Prédiction sur le test à conf 0,01 (une seule fois), puis bilan par classe à conf 0,25 : bonnes détections, mauvaises classes, fausses alarmes, ratés.
- Galeries des images avec le plus de fausses alarmes et de ratés.
- Recherche d'un seuil par classe (0,05 à 0,90) qui maximise la F1 sur la **validation**, classe par classe ; comparaison sur le **test** entre seuil unique et seuils par classe.
- Écriture de `seuils_par_classe.json`, lu par l'application C#.

`fonction` `score_image(nom, cat) → int  (interne à main)`

Nombre d'événements d'une catégorie donnée dans une image ; sert à trier les pires images.

`fonction` `f1(donnees, seuils) → (F1, précision, rappel)  (interne à main)`

Apparie toutes les images avec les seuils donnés et compte VP (bonnes détections), FP (fausses alarmes et mauvaises classes) et FN (ratés), puis calcule précision, rappel et F1.

etape6_bilan.py

Rôle : appliquer le modèle NEU à la tôle synthétique difficile (test de changement de domaine). **Produit :** `etape6_resultat.png`.

`fonction` `tole_difficile() → image`

Recrée la tôle de `etape6_limites.py` en 1200 × 600.

`fonction` `dernier_modele() → str`

Identique à celle de l'étape 5.

`fonction` `main()`

Découpe la tôle en vignettes de 200 px qui se chevauchent de moitié, fait prédire chaque vignette, décale les boîtes dans le repère de la tôle, fusionne les doublons par NMS (`cv2.dnn.NMSBoxes`), affiche les détections et dessine prédictions (rouge) et vrais défauts (vert).

5. 03 — Intégration C# : YoloOnnx

YoloOnnx/Program.cs

Rôle : inférence YOLO en C# sur une image, sans Python ; validation du prétraitement et du décodage. **Lancer :** `dotnet run -- best.onnx image.jpg`. **Produit :** `resultat_onnx.jpg`.

bloc `Chargement`

Vérifie les arguments, crée la session ONNX Runtime (une seule fois), lit le nom et la taille de l'entrée dans le modèle, lit les noms des classes, charge l'image en couleur.

bloc `Inférence et latence`

Un passage d'échauffement non chronométré, puis 20 passages chronométrés ; le résultat du dernier est gardé. La latence affichée concerne l'inférence seule (`session.Run`).

bloc `Décodage`

Sortie `[1, 4 + nb_classes, N]` : pour chaque candidat, meilleure classe et score ; rejet sous 0,25 ; boîte ramenée aux coordonnées de l'image d'origine (on retire le padding, on divise par l'échelle).

bloc `NMS et affichage`

NMS par classe (boîtes décalées de `classe × 4096`), puis affichage des détections triées par score, boîtes bornées à l'image, dessin et enregistrement.

fonction `static Pretraiter(Mat image, int cote) → (tenseur, échelle, décalX, décalY)`

Letterbox identique à Ultralytics : redimensionnement sans déformation, bandes grises 114, puis remplissage du tenseur `[1, 3, cote, cote]` pixel par pixel (BGR → RGB, /255). Version lisible, remplacée par une version par blocs dans `Detecteur.cs`.

fonction `static LireNomsDesClasses(InferenceSession session) → string[]`

Extrait les noms des classes des métadonnées `names` écrites par Ultralytics (expression régulière), triés par numéro ; à défaut, génère « classe 0 », « classe 1 »…

YoloOnnx/comparer_python.py

Rôle : la même prédiction en Python, comme référence. **Lancer :** `python comparer_python.py best.onnx image.jpg`.

fonction `main()`

Charge le **même fichier ONNX** avec Ultralytics, prédit (224 px, conf 0,25), mesure la latence moyenne sur 20 appels (prétraitement, inférence et décodage compris) et affiche les détections dans le même format que le C#. Attendu : mêmes classes, scores à 0,01 près, boîtes à 1 ou 2 px près.

6. 03 — Intégration C# : InspectionWpf

Application opérateur : caméra pylon (émulée), file producteur/consommateur, inférence ONNX, seuils par classe, journal CSV, interface WPF.

```
Caméra ─► SurImageRecue (thread pylon) ─► Channel (30) ─► BoucleTraitement ─► Detecteur.Detecter
                                    │ file pleine                        ├─► JournalDetections (CSV)
                                    └─► NON_INSPECTEE (CSV)              └─► ImageTraitee ─► MainWindow (15 i/s)
BoucleDeclenchement : trigger logiciel     BoucleSurveillance : mode dégradé + reconnexion
```

CameraService.cs

Rôle : tout ce qui touche à la caméra et au modèle, sans interface graphique. Communique avec l'interface uniquement par événements.

Types

`type` `enum EtatCamera { Arretee, Connexion, Acquisition, Degrade }`

État affiché par le voyant de l'interface.

`type` `record Frame(Id, Largeur, Hauteur, Pixels, Horodatage)`

Une image reçue : numéro, taille, copie des pixels, heure de réception (UTC, pour la latence).

`type` `record ResultatImage(Id, Largeur, Hauteur, Pixels, Moyenne, Latence, Detections, InferenceMs)`

Résultat envoyé à l'interface : image, luminosité moyenne, latence totale, défauts détectés, temps d'inférence.

`type` `record struct Stats(Recues, Traitees, NonInspectees, Ratees, Erreurs, AvecDefaut)`

Photo des compteurs à un instant donné.

Événements et propriétés

`événement` `ImageTraitee · Message · EtatChange`

Levés depuis les threads du service (jamais depuis le thread UI) : un résultat d'image, un message de journal, un changement d'état.

`propriétés` `CheminModele · DossierImages · DureeTraitementMs`

Modèle ONNX à charger (s'il est absent, traitement simulé) ; dossier d'images que l'émulateur doit « filmer » ; durée de traitement simulée quand il n'y a pas de modèle.

Méthodes

`fonction` `public Stats Statistiques()`

Lit les six compteurs de façon sûre entre threads (`Interlocked.Read`).

fonction `public void Demarrer()`

Charge le modèle une seule fois et ses seuils par classe ; trouve la caméra ; remet les compteurs à zéro ; crée la file bornée (30 images, ne supprime jamais rien d'elle-même) ; s'abonne aux événements pylon ; ouvre la caméra (et la libère si l'ouverture échoue) ; crée le journal CSV de la session ; lance les tâches de traitement, de surveillance et de déclenchement. Appelée hors du thread UI.

fonction `private void OuvrirEtDemarrer()`

Ouvre la caméra et applique toute la configuration : format Mono8 ; pour l'émulateur, taille 200 × 200, image de test coupée, dossier d'images et lecture de fichiers activée (avec une ligne de diagnostic) ; déclenchement logiciel (`FrameStart` , `TriggerMode On` , source `Software`) ; exposition ; démarrage de l'acquisition. Réutilisée à chaque reconnexion.

fonction `private void AppliquerExposition()`

Règle le temps d'exposition avec `ExposureTime` ou, sur les anciennes caméras, `ExposureTimeAbs` . Une valeur refusée est signalée sans planter.

fonction `public void DefinirCadence(double imagesParSeconde)`

Période entre deux déclenchements = 1000 / cadence ms, cadence bornée entre 0,5 et 60 i/s.

fonction `private async Task BoucleDeclenchement(CancellationToken ct)`

Toutes les `_periodeMs` , si la caméra est connectée et prête, envoie un trigger logiciel (sous verrou). Un échec est signalé et la boucle continue ; l'annulation y met fin. Simule le codeur d'une vraie ligne.

fonction `public void DefinirExposition(double microsecondes)`

Mémorise la valeur et l'applique tout de suite si la caméra est connectée (sinon elle sera appliquée à la reconnexion).

fonction `public void ForcerPanne()`

Demande à l'émulateur une image corrompue (`ForceFailedBuffer`) pour tester la gestion des acquisitions ratées.

fonction `public void SimulerDeconnexion()`

Déclenche le même chemin qu'une vraie perte de connexion (libère le sémaphore), en faisant échouer volontairement les 3 premiers essais de reconnexion : on voit le mode dégradé et le délai qui double (1, 2, 4 s).

fonction `private void SurImageRecue(sender, ImageGrabbedEventArgs e)  — PRODUCTEUR`

Appelée par pylon sur son thread à chaque image. Si l'acquisition a réussi : numéro d'image, **copie** des pixels (pylon réutilise son buffer), horodatage, dépôt dans la file sans attendre. Si la file est pleine : l'image est comptée « non inspectée », la zone est écrite dans le journal et un message est envoyé. Si l'acquisition a échoué : compteur « ratées » et message.

fonction `private async Task BoucleTraitement(ChannelReader<Frame> lecteur)  — CONSOMMATEUR`

Lit les images une par une (`await foreach`). Pour chacune : luminosité moyenne, inférence chronométrée (ou traitement simulé), compteurs ; s'il y a des défauts, ligne de journal avec la version du modèle et message ; puis envoi du résultat (avec latence totale) à l'interface. Un `try/catch` par image : une erreur est comptée et l'inspection continue. Se termine quand la file est fermée et vide.

fonction `private void ChargerSeuils(string chemin)`

Lit `seuils_par_classe.json` (à côté du modèle) et remplit les seuils par classe du détecteur.

fonction `private static double Moyenne(byte[] pixels)`

Luminosité moyenne de l'image (somme en `long`, division en `double`). Indicateur de santé de l'éclairage.

fonction `private void SurConnexionPerdue(sender, EventArgs e)`

Appelée par pylon quand la caméra ne répond plus : message et libération du sémaphore, sans rien faire de long.

fonction `private async Task BoucleSurveillance(CancellationToken ct)`

Dort sur le sémaphore ; à chaque perte de connexion, lance `Reconnecter`, puis se rendort. L'annulation y met fin.

fonction `private async Task Reconnecter(CancellationToken ct)`

Ferme proprement l'ancienne connexion (chaque appel dans son propre `try`), passe en mode dégradé et le signale, puis tente `OuvrirEtDemarrer` (sauf essais ratés simulés) avec un délai qui double à chaque échec (1, 2, 4… plafonné à 30 s). En cas de réussite : retour à l'état Acquisition.

fonction `public async Task ArreterAsync()`

Arrêt dans l'ordre du flux : annule et attend la surveillance et le déclenchement ; ferme la caméra ; ferme la file et attend que le consommateur ait traité les images restantes ; ferme le journal ; se désabonne des événements pylon et libère la caméra ; affiche le bilan de la session.

fonction `public void Dispose()`

Nettoyage final à la fermeture de l'application : annulation, caméra, session ONNX et journal.

Detecteur.cs

Rôle : inférence YOLO avec ONNX Runtime sur des images Mono8, prétraitement identique à Ultralytics.

type `record Detection(string Classe, float Score, Rect Boite)`

Un défaut détecté, boîte en pixels de l'image d'origine.

propriétés `Classes · Version · SeuilConfiance (0,25) · SeuilNms (0,7) · SeuilsParClasse`

Noms des classes lus dans le modèle ; version (nom, date du fichier, taille) écrite avec chaque décision ; seuils par défaut ; seuils par classe chargés depuis le JSON.

`fonction` `public Detecteur(string cheminOnnx)  — constructeur`

Crée la session ONNX Runtime (une seule fois), lit le nom et la taille de l'entrée dans le modèle (224 par défaut), lit les noms des classes et construit la version.

`fonction` `public List<Detection> Detecter(byte[] pixels, int largeur, int hauteur)`

Copie les pixels dans un `Mat` gris, passe en 3 canaux, applique le letterbox, lance l'inférence (`_session.Run`), choisit le décodeur selon la forme de la sortie, puis garde les détections au-dessus du seuil de leur classe, borne les boîtes à l'image et trie par score décroissant. Les `Mat` sont libérés avec `using`.

`fonction` `private DecoderAvecNms(Tensor s, echelle, dx, dy) → List<Detection>`

Sortie `[1, 4 + nc, N]` : pour chaque candidat, meilleure classe ; rejet sous le plus bas des seuils (pour ne rien couper avant les seuils par classe) ; retour aux coordonnées d'origine (inverse du letterbox) et passage centre → coin ; NMS par classe grâce à un décalage de `classe × 4096`.

`fonction` `private DecoderEndToEnd(Tensor s, echelle, dx, dy) → List<Detection>`

Sortie `[1, N, 6]` (x1, y1, x2, y2, score, classe) déjà filtrée par le modèle : ignore les lignes vides et ramène les boîtes aux coordonnées d'origine.

`fonction` `private static Letterbox(Mat bgr, int taille) → (tenseur, échelle, dx, dy)`

Échelle = plus petit des deux rapports (pas de déformation) ; redimensionnement ; bandes grises 114 réparties des deux côtés ; BGR → RGB ; division par 255 ; passage HWC → NCHW par plans (`Split` + `Marshal.Copy`). Renvoie le tenseur `[1, 3, taille, taille]` et les paramètres pour revenir aux coordonnées d'origine.

`fonction` `private static LireNomsDesClasses(InferenceSession) → string[]`

Identique à celle de `YoloOnnx`.

`fonction` `public void Dispose()`

Libère la session ONNX (mémoire native).

JournalDetections.cs

Rôle : un fichier CSV par session pour la traçabilité. Colonnes : horodatage ; image ; classe ; score ; x ; y ; largeur ; hauteur ; inference_ms ; modele.

`fonction` `public JournalDetections(string chemin)  — constructeur`

Crée le fichier avec `AutoFlush` (rien de perdu en cas d'arrêt brutal) et écrit l'en-tête.

`fonction` `public void Ecrire(long idImage, IEnumerable<Detection>, string versionModele, double inferenceMs)`

Une ligne par défaut détecté, avec le score, la boîte, le temps d'inférence et la version du modèle (format numérique invariant).

`fonction` `public void EcrireNonInspectee(long idImage, DateTime horodatageUtc, string versionModele)`

Une ligne `NON_INSPECTEE` pour une image reçue mais pas traitée (file pleine) : la zone doit être recontrôlée.

`fonction` `public void Dispose()`

Ferme le fichier. Toutes les écritures passent par un verrou, car le producteur et le consommateur y écrivent.

MainWindow.xaml et MainWindow.xaml.cs

Rôle : l'interface opérateur. Tout ce fichier s'exécute sur le thread UI, qui ne fait qu'afficher. **XAML :** image en direct à gauche ; à droite, voyant d'état, boutons Démarrer / Arrêter / Forcer une panne / Simuler déconnexion, curseurs de cadence (1–30 i/s), d'exposition (µs) et de traitement simulé (ms), compteurs et journal.

`bloc` `Chemins`

Dossier `donnees` à côté de l'exécutable : `best.onnx` et `images/`. Les variables `INSPECTION_MODELE` et `INSPECTION_IMAGES` permettent de pointer ailleurs.

`fonction` `public MainWindow()  — constructeur`

S'abonne aux événements du service : le dernier résultat est déposé avec `Interlocked.Exchange` ; messages et changements d'état sont renvoyés au thread UI par `Dispatcher.InvokeAsync`. Crée le minuteur d'affichage à 66 ms (~15 i/s).

`fonction` `Demarrer_Click · Arreter_Click`

Lancent `Demarrer` / `ArreterAsync` hors du thread UI (`await Task.Run`), gèrent l'état des boutons et du minuteur, et affichent une erreur de démarrage dans le journal.

`fonction` `Panne_Click · Deconnexion_Click · Cadence_Changed · Exposition_Changed · Traitement_Changed`

Transmettent l'action ou la valeur du curseur au service.

`fonction` `Window_Closing`

À la fermeture : arrêt du minuteur, arrêt propre du service, puis `Dispose`.

`fonction` `private void RafraichirAffichage()`

Récupère et vide le dernier résultat ; s'il y en a un, convertit l'image en couleur, dessine les boîtes et étiquettes, l'écrit dans un `WriteableBitmap` réutilisé, et mémorise latence, luminosité, temps d'inférence et classes détectées. Met ensuite à jour le texte des compteurs.

`fonction` `private void AjouterJournal(string message)`

Ajoute un message horodaté en tête de liste, en gardant au plus 200 lignes.

`fonction` `private void AfficherEtat(EtatCamera etat)`

Couleur du voyant et texte : vert « Acquisition en cours », orange « Connexion… », rouge « Mode dégradé », gris « Arrêtée ».

App.xaml et App.xaml.cs

Point d'entrée WPF standard : déclare `MainWindow` comme fenêtre de démarrage. Aucune logique.

7. 04 — Caméra motorisée

04_camera_motorisee/Program.cs

Rôle : séquence d'acquisition avec une caméra motorisée (moteur simulé, caméra émulée). **Lancer :** `dotnet run`.
Produit : dossier `inspection_<date>/` avec les images PNG et `journal.csv`.

`bloc` `Paramètres`

Plan d'inspection : 0°, 30°, 60°, 90°, 120°. Stabilisation 300 ms. Au plus 2 essais pour le moteur et 2 pour la capture.

`bloc` `Dossier et journal`

Un dossier horodaté par tournée et un journal CSV (horodatage ; position ; étape ; résultat ; fichier).

`bloc` `Caméra`

Vérifie qu'une caméra est disponible, configure le déclenchement logiciel et démarre l'acquisition sans boucle automatique : le programme ira chercher chaque image lui-même.

`bloc` `Tournée`

Inspecte chaque position, compte les réussites, ramène le moteur en position de repos ; un `finally` arrête et ferme toujours la caméra ; affichage du bilan (positions inspectées / total).

`fonction` `async Task<bool> InspecterPosition(double position)`

Machine à états d'une position (`while (true)` + `switch`) :

- **Deplacement** : moteur vers la position ; en cas de blocage, journal et nouvel essai, Erreur au 2e échec ;
- **Stabilisation** : attente de 300 ms (vibrations) ;
- **Capture** : caméra prête ? trigger, puis `RetrieveResult` (5 s maximum) ; en cas d'échec, nouvel essai ou Erreur ;
- **Controle** : contrôle qualité ; image refusée → libérée, recapture ou Erreur ;
- **Sauvegarde** : PNG `pos_XXX.png`, ligne de journal, libération de l'image ;
- **Termine** → `true` ; **Erreur** → libération, journal « abandon », `false`.

`fonction` `(bool, string) ControlerQualite(IGrabResult img)`

Refuse une image ratée, vide, trop sombre (moyenne < 20) ou trop claire (> 235) ; simule une image floue une fois sur cinq pour tester la recapture. Renvoie le verdict et sa raison.

`fonction` `void Journaliser(position, etape, resultat, fichier)`

Ajoute une ligne au journal, horodatée à la milliseconde.

`type` `enum Etape { Deplacement, Stabilisation, Capture, Controle, Sauvegarde, Termine, Erreur }`

Les états de la séquence.

`type` `class MoteurSimule`

Position : lisible par tous, modifiable par le moteur seul. **AllerAAsync(double cible)** : se bloque dans 15 % des cas (exception) pour tester la gestion d'erreur ; sinon attend un temps proportionnel à la distance (60 °/s) puis met à jour la position.